Comparing First-Visit Monte Carlo and Sarsa(λ) on Text Flappy Bird

Julian

CentraleSupélec — 3MD3220 Reinforcement Learning, April 2026

Abstract. We implement and compare two tabular reinforcement learning agents — First-Visit Monte Carlo (MC) and Sarsa(λ) — on the `TextFlappyBird-v0` environment. After 25 000 training episodes Sarsa(λ) achieves a mean score of **199 pipes** vs. **65 pipes** for MC, demonstrating the advantage of online, trace-based updates. We report learning curves, value-function heatmaps, policy maps, parameter sweeps, and a generalisation study across environment configurations.

1 Experimental Setup

Environment. `TextFlappyBird-v0` [2] returns a 2-tuple $(x_{\text{dist}}, y_{\text{dist}})$ — the player's horizontal and vertical distance to the nearest pipe gap. With the default config (`height`=15, `width`=20, `pipe_gap`=4) the discrete state space contains $14 \times 22 = 308$ states; the action space is binary (0 = idle, 1 = flap). Every step survived gives reward +1; episodes terminate on collision or after 2 000 steps.

Hyperparameters. Both agents share: $\alpha = 0.1$, $\gamma = 0.99$, initial $\varepsilon = 0.1$ (decaying to 0.01), 25 000 episodes. Sarsa additionally uses $\lambda = 0.7$.

2 Agent Implementations

First-Visit MC. MC collects a full episode, then updates Q only at the *first* visit to each (S_t, A_t):

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[G_t - Q(S_t, A_t)], \quad G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

No bootstrapping is used, so updates are unbiased but high-variance.

Sarsa(λ) [1]. At every step Sarsa computes a TD-error δ_t and propagates credit via eligibility traces $E(s, a)$:

$$\begin{aligned} \delta_t &= R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t), \\ E(S_t, A_t) &\leftarrow E(S_t, A_t) + 1, \\ Q(s, a) &\leftarrow Q(s, a) + \alpha\, \delta_t\, E(s, a), \quad E(s, a) \leftarrow \gamma\lambda\, E(s, a). \end{aligned}$$

Online updates and multi-step credit assignment lead to much faster convergence.

3 Results

Learning curves (Fig. 1). Sarsa(λ) (teal) rises steeply and plateaus near 2 000 reward steps around episode 15 000. MC (orange) converges slowly with high variance, consistent with its end-of-episode, no-bootstrapping design. Final greedy evaluation over 500 episodes: **Sarsa 199.0 pipes / 2 000 steps** vs. **MC 65.2 pipes / 662 steps**.

Value functions (Fig. 2). Both agents learn that small $|y_{\text{dist}}|$ (vertically aligned with the gap) is highly valuable. Sarsa's heatmap is smoother and more complete (298/308 states visited vs. 271/308 for MC), reflecting better credit propagation.

Policy maps (Fig. 3). Both agents converge to the intuitive strategy: *flap* when below the gap ($y_{\text{dist}} > 0$), *idle* when above. Sarsa covers nearly every state; MC leaves corner states at default Q = 0.

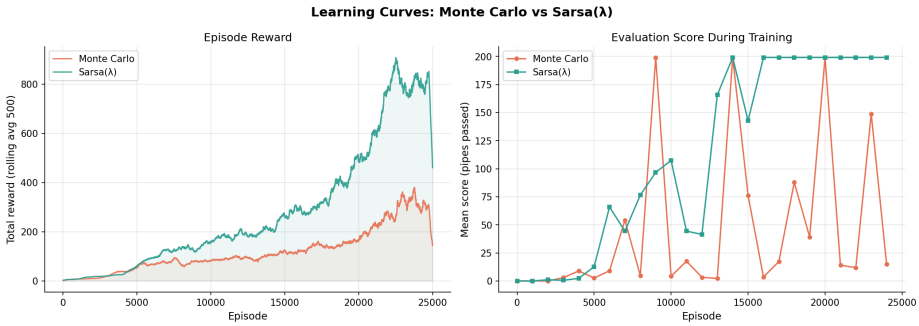

Fig. 1. Learning curves: rolling reward (left) and pipe score (right) over 25 000 episodes.

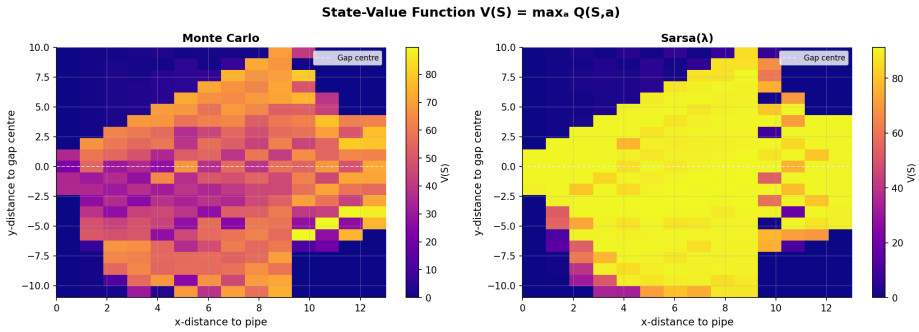

Fig. 2. State-value heatmaps $V(s) = \max_a Q(s, a)$.

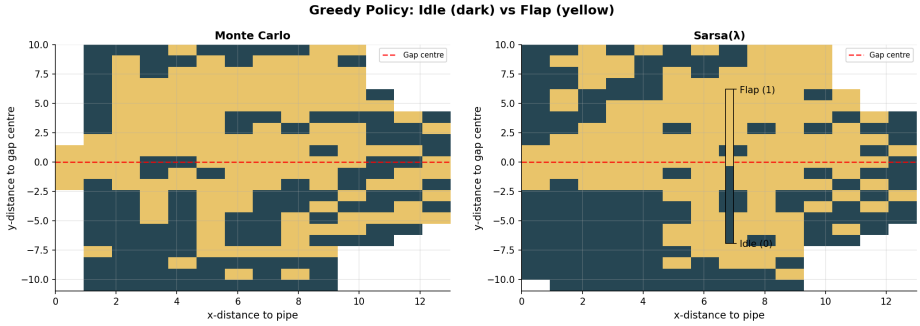

Fig. 3. Greedy policy maps (blue = flap, orange = idle).

4 Discussion

Q1 — Algorithm comparison

Table 1 summarises the key differences. MC is simpler but slow: short early episodes provide little signal per episode refresh. Sarsa(λ) bootstraps and spreads credit backwards at every step, achieving both faster convergence and higher final performance. Sensitivity-wise, MC is primarily sensitive to α; Sarsa adds λ as a critical parameter (see Sect. 4).

Table 1. Agent comparison.

	Monte Carlo	**Sarsa(λ)**
Update timing	End of episode	Every step
Bootstrapping	No	TD error
Traces	No	λ-weighted
States explored	271 / 308	298 / 308
Mean score (25k)	65.2 pipes	**199.0 pipes**
Key parameters	α	α, λ

Q2 — Two TFB environment versions

`TextFlappyBird-v0` gives a compact 2D distance observation, yielding 308 discrete states — ideal for tabular methods. `TextFlappyBird-screen-v0` returns the full ASCII screen render: the state space grows exponentially with grid size (curse of dimensionality), making tabular Q-tables intractable. Use-case limitation: the compact version loses information on bird velocity and multiple upcoming pipes; the screen version is faithful but requires function approximation (e.g. DQN with a CNN).

Q3 — Original Flappy Bird

The original `flappy-bird-gym` [3] uses continuous physics (gravity, velocity) and pixel observations. Tabular agents cannot scale to continuous or high-dimensional spaces without discretisation. In practice one would use *tile-coding* for state aggregation, or a deep agent such as DQN with frame stacking and convolutional feature extraction.

Q4 — Generalisation across configurations

We trained both agents on the default config and evaluated them on an alternative configuration (Fig. 4; Table 2). Zero-shot transfer degrades performance sharply: a new config shifts the (x, y) state distribution, exposing unseen states whose Q-values default to zero. Retraining Sarsa(λ) from scratch on the new config recovers near-perfect play (**249 pipes**). This highlights the fundamental limitation of tabular agents: they memorise state-action pairs and do not generalise across environment geometries.

Table 2. Generalisation results (mean score / mean reward).

Condition	Score	Reward
MC (default → alt)	12.0	106.3
Sarsa(λ) (default→alt)	1.7	23.6
MC retrained on alt	4.9	48.1
Sarsa(λ) retrained	**249.0**	**2000.0**

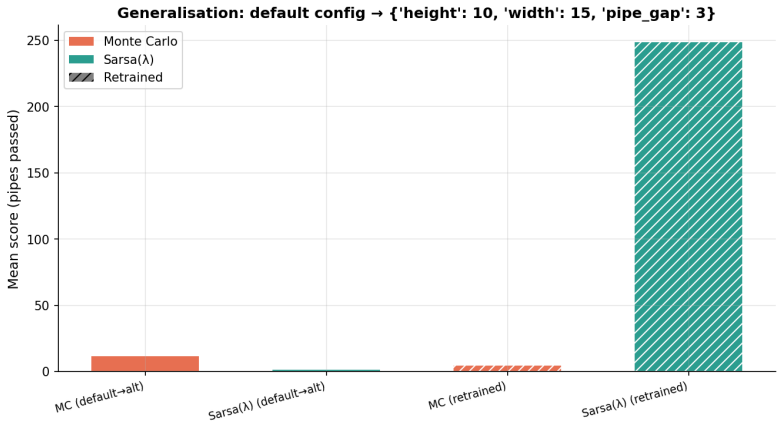

Fig. 4. Generalisation: mean score across four conditions.

Q5 — Parameter sensitivity

Figure 5 shows sweeps over α and λ (10 000 episodes each). α **sweep:** MC peaks at $\alpha = 0.1$ (score 153.6 at 10k ep.); Sarsa at $\alpha = 0.2$ (score 199.0). Very small α ($\leq$0.05) prevents convergence; large α ($\geq$0.5) causes instability. λ **sweep:** Performance increases monotonically up to $\lambda = 0.9$ (score 53.6 at 10k ep.) then drops slightly at $\lambda = 0.99$ due to near-infinite trace accumulation. Both agents are fairly robust around their optimal values.

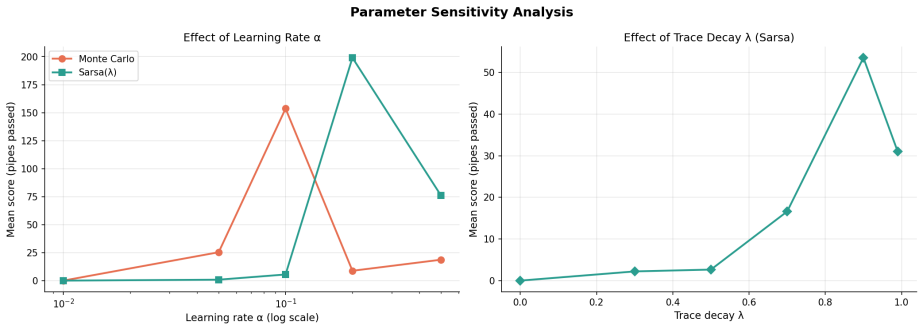

Fig. 5. α sweep for both agents (left) and λ sweep for Sarsa(λ) (right) at 10 000 training episodes.

5 Conclusion

Sarsa(λ) substantially outperforms First-Visit MC on `TextFlappyBird-v0` (199 vs. 65 mean pipes after 25 000 episodes), thanks to online, bootstrapped updates with eligibility traces. Both agents are interpretable through their tabular Q-functions, but neither generalises across environment configurations without retraining. Future work could explore linear function approximation or DQN to handle larger state spaces and enable configuration transfer.

Code notebook: https://github.com/YOUR_USERNAME/flappy-rl-assignment

References

1. R. Sutton, A. Barto: *Reinforcement Learning: An Introduction*, 2nd edn. MIT Press (2018).
2. S. Christodoulidis: Text Flappy Bird Gym. GitLab (2023). https://gitlab-research.centralesupelec.fr/stergios.christodoulidis/text-flappy-bird-gym
3. Talendar: Flappy Bird Gym. GitHub (2021). https://github.com/Talendar/flappy-bird-gym